

WRO 2026 - FUTURE ENGINEERS

X-Cute Team- Documentary

[GitHUB](#)

Aghiad Jomah (coach)

Mohamad Alzoubi

Elyas Kuzma

المحتويات

4.....	Introduction	
5.....	Our Team	1.
6.....	Our Robot	2.
6.....	Top Layer (Third Layer):	2.1.
6.....	Middle Layer (Second Layer):	2.2.
6.....	Bottom Layer (First Layer):	2.3.
7.....	Mobility Management	3.
7.....	Vehicle Main Body	3.1.
7.....	3D-Designed Parts	3.1.1.
8.....	Motion Mechanism	3.2.
10.....	Power and Sense Management	4.
11.....	Sensors	4.1.
11.....	Laser Sensors (VL53L0X - ToF Sensors):	4.1.1.
12.....	Ultrasonic Sensors (US-100):	4.1.2.
12.....	Inertial Measurement Unit (BNO085 IMU):	4.1.3.
12		
13.....	Camera (FANTECH LUMINOUS C30):	4.1.4.
14.....	Motor Drivers	4.2.
14.....	DC Motor Driver (IBT-4):	4.2.1.
14.....	Servo Driver (Waveshare Serial Bus	4.2.2.
14	Servo Driver):	
15.....	Controllers and Microprocessors	4.3.
15.....	Raspberry Pi 5 (8GB RAM):	4.3.1.
16.....	STM32F411 Microcontroller (BlackPill):	4.3.2.
17.....	Power Management	4.4.
21.....	Obstacle Management	5.
21.....	System Architecture & Communication	5.1.
21.....	Open Challenge	5.2.
21.....	"Direction Lock" Algorithm:	5.2.1.
21.....	Image Processing	5.3.
21.....	Region of Interest & Contouring:	5.3.1.
22.....	Obstacle Challenge	5.4.
22.....	Performance Videos	5.5.
22.....	Problems We Encountered	5.6.

24..... Improvements and future modifications

6.

25..... 7. Conclusion

Introduction

In this report, we present our self-driving system developed for the WRO 2026 - Future Engineers category. Our robot bridges advanced computer vision for strategic decision-making with real-time motion control. The vehicle features an Ackermann steering mechanism and a hybrid sensory system combining the precision of Laser sensors for highly accurate distance measurement and maneuvering, alongside Ultrasonic sensors that act as a fast-response safety system for distance verification. High-level visual perception and processing are powered by a Raspberry Pi, while an STM32 microcontroller executes real-time PID control algorithms. The software architecture is engineered to ensure seamless navigation, dynamic obstacle dodging, and precise final parking.



1. Our Team

We are team X-CUTE. Our team represents a product of technical dedication and engineering passion, focused on integrating advanced Embedded Systems engineering with Computer Vision algorithms to build self-driving vehicles capable of analyzing their environment and making independent, reliable navigational decisions in dynamic environments.



2. Our Robot

Our robot is a four-wheeled autonomous vehicle built on a robust, custom-designed chassis modeled in SolidWorks. The structure consists of three main layers to ensure optimal weight distribution, efficient organization of electronic and mechanical components, and streamlined cable management:

2.1. Top Layer (Third Layer):

Serves as the real-time control and vision platform. This layer houses the custom Printed Circuit Board (PCB), the STM32 microcontroller responsible for executing low-level control algorithms, and the camera module. Elevating the camera to this level ensures a clear, unobstructed, and wide Field of View (FoV) for highly accurate track and pillar detection.

2.2. Middle Layer (Second Layer):

Acts as the high-level processing and decision-making hub. This layer hosts the Raspberry Pi, which handles computer vision and complex data processing, alongside the Motor Driver.

2.3. Bottom Layer (First Layer):

Dedicated to the drivetrain, steering mechanism, and power source. This arrangement ensures a low Center of Gravity (CoG) to enhance vehicle stability during fast maneuvers and sharp cornering. It contains the main drive motor, the smart steering actuator (Servo), and the main battery that powers the entire system.

3. Mobility Management

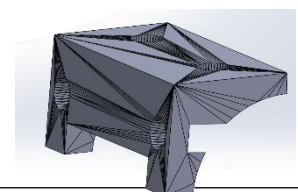
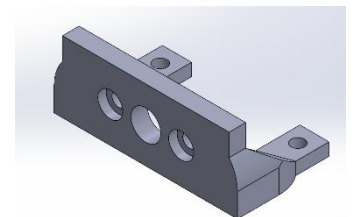
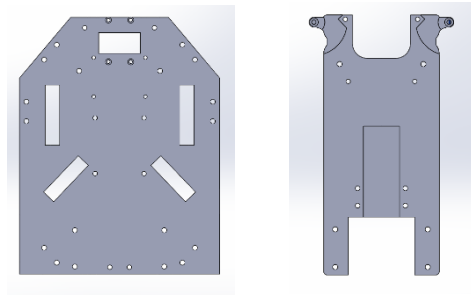
3.1. Vehicle Main Body

The robot relies on a robust chassis entirely precision-designed using SolidWorks. The structure is divided into three functional layers, which were fully manufactured using 3D printing technology to ensure optimal weight distribution, mechanical isolation, and ease of wiring for the custom Printed Circuit Board (PCB). The design focuses on balancing durability and lightweight properties, allowing the robot to withstand vibrations from rapid movements.

3.1.1. 3D-Designed Parts

To ensure seamless integration and minimize overall weight, all critical mechanical components, including the main body layers, were custom-designed and manufactured using 3D printing technology. Each part was meticulously engineered to serve a specific function within the robot's ecosystem:

- The Three Main Chassis Layers (Top, Middle, Bottom):** 3D printing the main body allowed us to create a custom, lightweight structure with pre-designed mounting holes, integrated standoffs, and optimized cable routing paths that
 Ackermann Steering Joints: Custom-designed to achieve the exact Ackermann geometry required for our wheelbase. 3D printing these knuckles allowed us to optimize the pivot points, ensuring maximum steering angles and precise cornering without wheel scrub.
- DC Motor Mount:** Engineered to firmly secure the Ga25 main drive motor to the bottom layer. It absorbs operational motor shaft and the differential gear to prevent any power loss.
- Ultrasonic Sensor Holders:** Specially designed brackets to mount the US-100 sensors to the sides and front of the height and angle to detect track walls effectively while avoiding false sound-wave reflections from the floor.
- Laser Sensor Mounts:** Custom mounts created to hold the guarantees highly accurate targeting and distance



measurement for the colored pillars during the Obstacle Challenge.

3.2. Motion Mechanism

3.2.1. Ackermann Steering Mechanism:

To ensure smooth and precise responsiveness during turns, our vehicle employs Ackermann steering geometry. This system allows the inner wheel to steer at a sharper angle than the outer wheel, minimizing tire slip and scrub while maintaining an optimal center of rotation. To mechanically actuate this system, we integrated the DYNAMIXEL XL330 smart servo motor. This actuator is characterized by its high-resolution positioning, consistent torque, and serial communication (UART) support, enabling it to receive highly accurate and instantaneous steering angles directly from the STM32 microcontroller.

EQUATIONS

- Let w be the track width, l the wheel base, ϕ_i the relative steering angle of the inner wheel, ϕ_o the relative steering angle of the outer wheel, and r the distance between ICC (instantaneous center of curvature) and the center of the car, we can write the following:

- $\tan \phi = \frac{l}{r}$

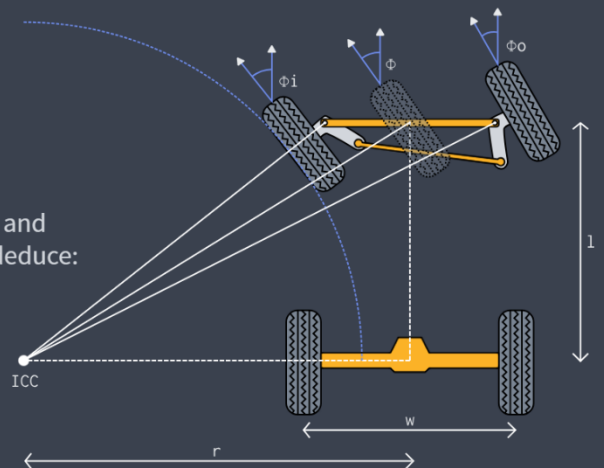
- $\tan \phi_i = \frac{l}{r - w/2}$

- $\tan \phi_o = \frac{l}{r + w/2}$

- By subtracting the reciprocal of these equations, and considering the fact that $\cot x = \frac{\cos x}{\sin x}$, we can deduce:

- $\phi_i = \tan^{-1} \left(\frac{2l \cdot \sin \phi}{2l \cdot \cos \phi - w \sin \phi} \right)$

- $\phi_o = \tan^{-1} \left(\frac{2l \cdot \sin \phi}{2l \cdot \cos \phi + w \sin \phi} \right)$

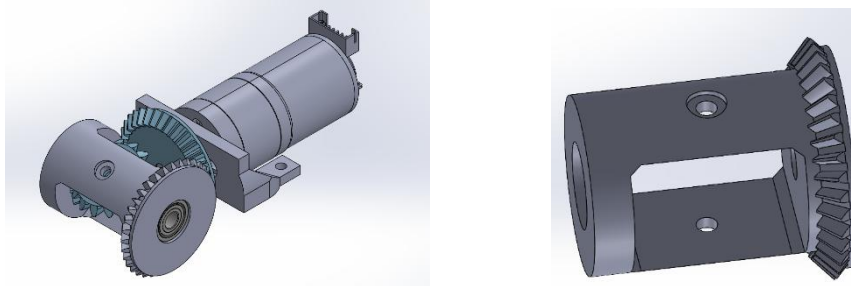


3.2.2. Driving Motor:

The vehicle is powered by a Ga25 DC motor operating at 12 Volts. Equipped with a gearbox, it delivers a rotational speed of 600 RPM, providing the robot with an excellent balance between torque and the top speed required to traverse the track sections efficiently.

3.2.3. Differential Gear:

To ensure a smooth distribution of kinetic energy to the rear wheels, we implemented a Differential Gear connecting the motor to the drive axle. This mechanism allows the outer wheel to rotate faster than the inner wheel while cornering, preventing the robot from skidding, improving maneuverability, and maintaining movement stability.



3.2.4. Wheels:

The robot is equipped with Silicone wheels. This material provides exceptional grip and traction on the track surface, minimizing slippage and translating the motor's torque into effective acceleration and immediate active braking response.

3.2.5. Driving Motor: This is the DC-motor we are using in the robot. It is connected to the differential mechanism to ensure a smooth distribution of kinetic energy to the rear wheels.

Ga25 DC Motor (12 V, 600 RPM)	
Type: Brushed DC gear motor with metal gearbox	
Nominal Voltage: 12 V DC	
Gear Ratio: $\approx 10:1$	
No-Load Speed: ≈ 600 RPM ; No-Load Current: ≈ 100 mA	
Torque: Continuous ≈ 0.4 kg·cm , Stall ≈ 1.5 kg·cm	
Stall Current: ≈ 1.2 A	
Mechanical: Diameter ≈ 25 mm ; 4 mm D-shaft ; Weight ≈ 85 g	
Recommended Driver: IBT-4 Motor Driver (High-current MOSFET capable of handling Back EMF)	
Notes: Provides an optimal balance of top speed for the straight track sections and sufficient torque to drive the differential gear efficiently.	

Table 1: Ga25 DC Motor (12 V, 600 RPM) – simplified datasheet view.

3.3. Steering Motor: For the steering mechanism, we used the DYNAMIXEL XL330 smart servo due to its high-resolution positioning, consistent torque, and ability to provide real-time data feedback. Unlike standard RC servos that rely on simple PWM signals, this smart actuator communicates via a serial bus, allowing the STM32 microcontroller to send highly accurate angular commands and read the motor's exact position.

DYNAMIXEL XL330 Smart Servo	
Type: Smart Actuator / Serial Bus Servo	
Operating Voltage: 3.7 V – 6.0 V (Operated at 5 V)	
Stall Torque: $\approx 5.3 \text{ kg}\cdot\text{cm}$ (0.52 N·m at 5 V)	
Speed: $\approx 57 \text{ RPM}$ (0.17 s / 60° at 5 V)	
Control Signal: Half-Duplex Asynchronous Serial (UART / TTL)	
Feedback & Features: Internal PID controller ; Real-time feedback for Position, Velocity, Load, and Temperature	
Mechanical: Dimensions $\approx 20.0 \times 34.0 \times 26.0 \text{ mm}$; Weight $\approx 18 \text{ g}$; Engineering Plastic Gears	
Recommended Driver: Waveshare Serial Bus Servo Driver (Converts standard UART to Half-Duplex Serial)	

Table 2: DYNAMIXEL XL330 Smart Servo – simplified datasheet view.

3.4. Engineering Principles

3.4.1. Weight Distribution & CoG:

Components were strategically placed so that the heaviest parts (the Ga25 main motor, the DYNAMIXEL servo, and the battery) are located on the first (bottom) layer. This design ensures a low Center of Gravity (CoG), preventing the robot from tipping over during sharp maneuvers and maximizing the silicone tires' traction on the surface.

4. Power and Sense Management

This section explains how power consumption was managed across the robot's electrical components and details the hybrid sensing system used for data acquisition.

4.1. Sensors

To navigate safely and at high speeds, we integrated multiple sensing technologies to cover all blind spots and provide comprehensive environmental data:

4.1.1. Laser Sensors (VL53L0X - ToF Sensors):

These Time-of-Flight sensors provide highly accurate, millimeter-level distance measurements. They feature a rapid response time and immunity to ambient light variations, making them our primary sensors for precise, dynamic pillar dodging.

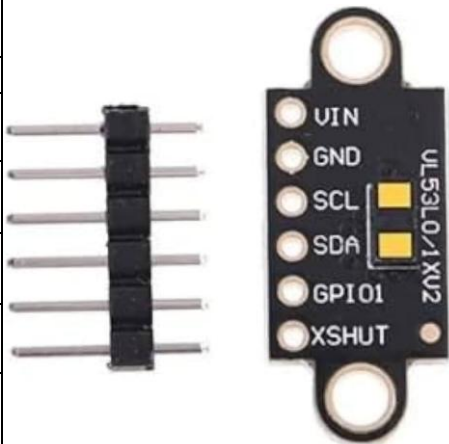
VL53L0X Time-of-Flight (ToF) Sensor	
Type: Laser-ranging ToF sensor (940 nm VCSEL emitter)	
Range: Up to 200 cm (2 meters)	
Resolution: 1 mm ; Accuracy: $\pm 3\%$; Measurement Rate: Up to 50 Hz (High-speed mode)	
Interface: I2C (up to 400 kHz)	
Timing: Adjustable timing budget (typically 20 ms to 33 ms per reading)	
Supply Voltage: 2.6 V – 3.5 V (Breakout boards accept 3.3V or 5V) ; Current: ≈ 20 mA (active)	
Field of View: 25° Field of Illumination (FoI)	
Size: Module size typically $\approx 10 \times 13$ mm (varies by breakout board)	

Table 3: VL53L0X Time-of-Flight Sensor – simplified datasheet view.

4.1.2. Ultrasonic Sensors (US-100):

These act as a robust safety net and wall-following system. The US-100 features a built-in temperature sensor for the "Temperature Compensation" of the speed of sound. This ensures highly stable distance readings regardless of the heat generated by the motors or the ambient track temperature.

US-100 Ultrasonic Sensor	
Type: Ultrasonic distance sensor with active temperature compensation	
Range: 2 cm – 450 cm	
Resolution: 1 mm ; Accuracy: High (Temperature Compensated)	
Interface: GPIO (Trig/Echo) or UART mode	
Timing: 10 μ s trigger pulse ; max echo timeout $\approx$ 26 ms	
Supply Voltage: 2.4 V – 5.5 V ; Current: < 2 mA (quiescent)	
Field of View: $\approx$ 15° beam angle	
Size: 45 × 20 × 15 mm	

Table 4: US-100 Ultrasonic Sensor – simplified datasheet view.

4.1.3. Inertial Measurement Unit (BNO085 IMU):

As mentioned earlier, we rely on this high-precision 9-DoF sensor to accurately estimate the target heading without suffering from electromagnetic interference.

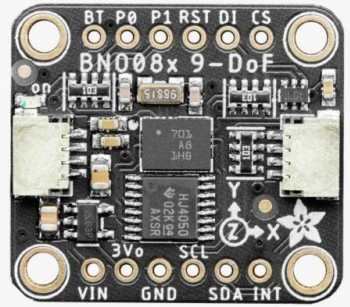
BNO085 9-DoF IMU (Inertial Measurement Unit)	
Type: 9-Axis System-in-Package (SiP) with integrated Sensor Fusion processor	
Sensors: 3-axis Accelerometer, 3-axis Gyroscope, 3-axis Magnetometer	
Output Data: Quaternions, Euler Angles (Roll, Pitch, Yaw), Linear Acceleration, Gravity Vector	
Interface: I2C (up to 400 kHz), SPI, and UART	
Heading Accuracy: $\approx$ 1.5° (Static) ; $\approx$ 3.5° (Dynamic)	
Operating Voltage: 3.3 V (Most breakout boards include a regulator for 3.3V – 5V operation)	
Mechanical: Module size typically $\approx$ 20 × 25 mm (varies by breakout board)	

Table 5: BNO085 9-DoF IMU – simplified datasheet view.

4.1.4. Camera (FANTECH LUMINOUS C30):

Instead of standard ribbon-cable cameras, we opted for a high-definition USB web camera connected directly to the Raspberry Pi. This camera provides a wide field of view, crisp resolution (2K), and seamless plug-and-play integration with ROS and OpenCV to detect lines and pillars with utmost clarity.

FANTECH LUMINOUS C30 Web Camera	
Type:	High-Definition USB Web Camera
Maximum Resolution:	2K (Quad HD – 2560 × 1440 pixels)
Frame Rate:	Up to 30 FPS
Field of View (FOV):	Ultra-Wide Angle (approx. 106°)
Interface:	USB 2.0 (Plug-and-Play)
Lens:	High-quality optical glass lens
Mechanical:	Features a 360-degree rotating base, allowing for the precise tilt-down angle required when mounted on the robot's third layer.



Table 6: FANTECH LUMINOUS C30 Web Camera – simplified datasheet view.

4.2. Motor Drivers

4.2.1. DC Motor Driver (IBT-4):

To control our powerful Ga25 drive motor, we selected the IBT-4 driver, which utilizes heavy-duty MOSFETs capable of handling extreme currents. This driver prevents voltage drops during active braking or sudden acceleration and protects the circuit from significant Back EMF.

IBT-4 DC Motor Driver	
Type: High-current MOSFET H-Bridge DC motor driver	
Operating Voltage: 5 V – 15 V (Perfectly accommodates the 11.1 V 3S Li-Po battery)	
Current Capacity: Up to 50 A peak (Provides massive headroom for the Ga25 motor's 1.2 A stall current, preventing any thermal stress)	
Control Signal: PWM inputs for speed and direction control ; Supports high PWM frequencies (up to 200 kHz)	
Logic Level: 3.3 V and 5 V compatible (Directly interfaced with the STM32F411 3.3 V logic without level shifters)	
Protection Features: Over-voltage, under-voltage, over-temperature, and short-circuit protection	
Mechanical: Equipped with a large aluminum heat sink for efficient passive thermal dissipation	

Table 7: IBT-4 DC Motor Driver – simplified datasheet view.

4.2.2. Servo Driver (Waveshare Serial Bus Servo Driver):

To allow the STM32 to communicate with the DYNAMIXEL XL330 smart servo, we incorporated this specific driver board. It converts standard UART signals into the required half-duplex serial bus and provides clean, isolated power routing for the servo.

Waveshare Serial Bus Servo Driver	
Type: UART to Half-Duplex Serial Bus Converter	
Logic Level: 3.3 V and 5 V compatible (Directly interfaces with the STM32F411 without level shifters)	
Power Supply (Servo): Wide input voltage range (Power is routed directly to the servo, completely isolating motor current from the logic circuits)	
Input Interface: Standard Full-Duplex UART (RX / TX / GND)	
Output Interface: Half-Duplex Single-Wire Serial (DATA / VCC / GND)	
Switching Mechanism: Hardware-level automatic TX/RX direction switching (Eliminates the need for a dedicated GPIO pin to control communication direction)	

Table 8: Waveshare Serial Bus Servo Driver – simplified datasheet view.

4.3. Controllers and Microprocessors

Our system relies on a Hybrid Control Architecture that separates high-level processing tasks from low-level real-time motion control to ensure ultra-fast, zero-latency responsiveness.

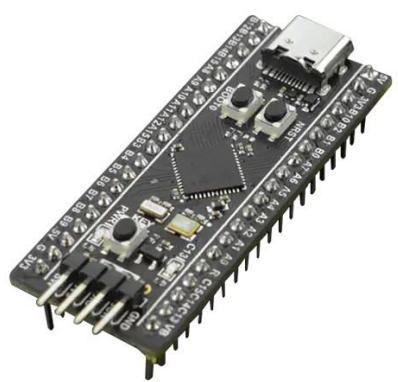
4.3.1. Raspberry Pi 5 (8GB RAM):

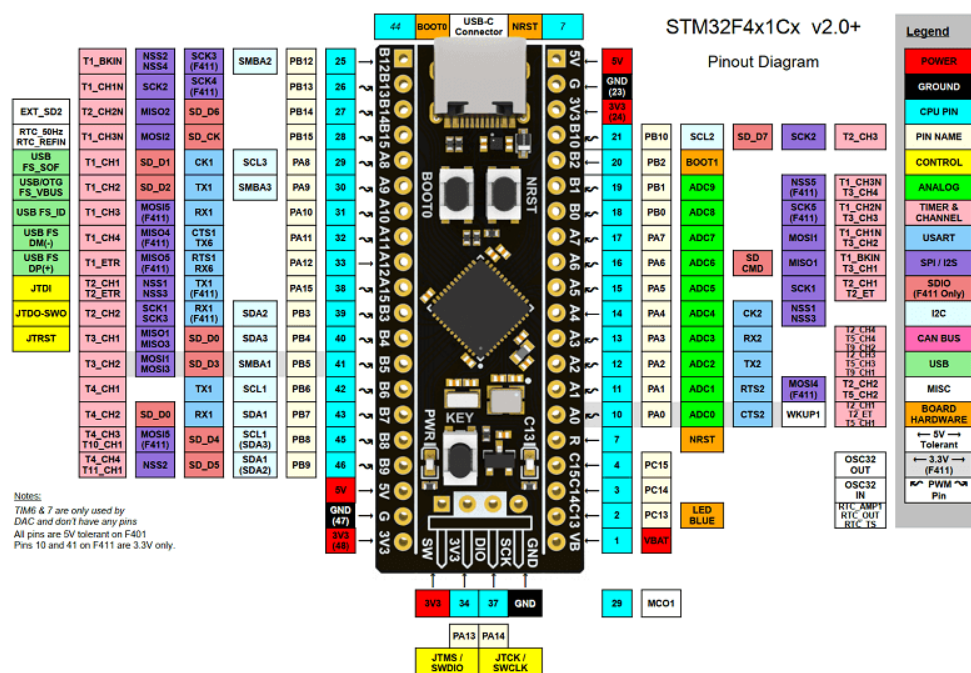
- 4.3.1.1. System Role:** Acts as the brain of the robot. It runs the operating system and executes the main control scripts (Python), alongside complex computer vision algorithms via OpenCV. It is responsible for processing camera frames, applying the Y-Axis depth-sorting algorithm for the red and green pillars, and subsequently sending strategic commands directly to the STM32 via serial communication (UART).
- 4.3.1.2. Technical Specifications:** Powered by a Broadcom BCM2712 quad-core Cortex-A76 64-bit processor clocked at 2.4GHz, featuring a massive 8GB LPDDR4X RAM, along with ultra-fast high-speed I/O interfaces.
- 4.3.1.3. Justification:** The Pi 5 delivers a massive leap in processing power compared to its predecessors. This high performance allows the robot to process 2K resolution images at high frame rates (FPS) without any computational bottlenecks, ensuring that dodging and cornering decisions are made in milliseconds and transmitted instantly over serial.

Raspberry Pi 5 (8GB RAM)	
Type: High-Performance Single Board Computer (SBC) / Master Node	
Processor (CPU): Broadcom BCM2712, Quad-core 64-bit Arm Cortex-A76 @ 2.4GHz	
Memory (RAM): 8GB LPDDR4X-4267 SDRAM (Provides massive headroom for high-resolution OpenCV image processing)	
Operating Voltage: 5.0 V DC via USB-C	
Key Interfaces: 2 × micro-HDMI, 2 × USB 3.0, 2 × USB 2.0, GPIO Header (utilizing UART TX/RX for STM32 communication)	
Cooling Requirement: Active Cooler (Heatsink + Fan) is highly recommended to prevent thermal throttling during continuous computer vision tasks	
Mechanical: Standard Pi form factor (85 × 56 mm)	

4.3.2. STM32F411 Microcontroller (BlackPill):

- 4.3.2.1. System Role:** Serves as the muscular and operational node. It receives strategic steering commands from the Raspberry Pi via UART and translates them into precise PWM signals for the motors. It also handles the hardware interrupts for the ultrasonic sensors and acquires data from the IMU.
- 4.3.2.2. Technical Specifications:** Built on an ARM Cortex-M4 core running at 100 MHz, equipped with a Hardware Floating-Point Unit (FPU), 512KB of Flash memory, and 128KB of SRAM.
- 4.3.2.3. Justification:** Advanced operating systems like Linux (used on the Pi) are not designed for hard real-time tasks. The STM32 provides Deterministic Execution, ensuring that PID calculations, motor speed updates, and sensor readings occur with microsecond precision without jitter. Additionally, the integrated FPU significantly accelerates the complex floating-point decimal math required for the real-time control algorithms.

STM32F411 Microcontroller (BlackPill)	
Type: 32-bit ARM Cortex-M4 Development Board	
Processor (Core): ARM Cortex-M4 with Hardware Floating-Point Unit (FPU) @ 100 MHz	
Memory: 512 KB Flash memory ; 128 KB SRAM	
Operating Voltage: 3.3 V Logic Level (Board is safely powered via 5V from the XL4015 converter)	
Key Interfaces: Multiple UART, I2C, and SPI buses ; Advanced hardware timers for highly accurate PWM generation ; External Interrupts (EXTI)	
Mechanical: Compact DIP form factor (approx. 53 × 21 mm)	



4.4. Power Management

We structured the electrical system based on the philosophy of "Separation of Power Domains" to guarantee absolute system stability:

Component	Quantity	Supply Voltage	Idle Current	Active Current	Peak/Stall Current
Raspberry Pi 5 (8GB)	1	5.0 V	~600–800 mA	~1.5–2.0 A	up to 5.0 A
FANTECH LUMINOUS C30	1	5 V (via Pi USB)	–	~150–250 mA	~300 mA
STM32F411 (BlackPill)	1	3.3 V / 5 V	~10 mA	~25–35 mA	~50 mA
BNO085 IMU	1	3.3 V	~3 mA	~10–15 mA	–
US-100 Ultrasonic Sensor	3	5.0 V	< 2 mA	~3–5 mA	–
VL53L0X Laser Sensor	2	3.3 V – 5.0 V	< 5 μ A	~20 mA	~40 mA
DYNAMIXEL XL330 Servo	1	5.0 V	~15 mA	~150–300 mA	~1.5 A
Ga25 DC Motor	1	12 V	–	~100–400 mA	~1.2 A
IBT-4 Motor Driver	1	5 V (Logic) / 12 V (Load)	~2 mA	Adds driver losses to motor current	Handles up to 50 A peak
Waveshare Servo Driver	1	5.0 V	~2 mA (logic)	Servo load passes through	–
XL4015 DC-DC Converter	1	12.0 V (in) / 5.0 V (out)	~5 mA	Load dependent	Handles up to 5 A peak

Table 11: Electrical Component Current Specifications

4.4.1. Logic Power:

The Raspberry Pi and its connected camera are powered by a dedicated 20W Power Bank. This complete isolation prevents the operating system from suffering voltage drops or "brownouts" caused by sudden, massive current draws from the motors.

4.4.2. Actuator Power:

We utilize a Lithium-Polymer battery (Li-Po 3S 11.1V 2200mAh 30C) as the primary power source for the lower layer. The high discharge rate (30C) effortlessly supplies the necessary burst current for the DC motor and servo during critical moments (e.g., initial acceleration or sudden stops) without voltage sagging.

Battery:

Table 13: Li-Po 3S Battery (11.1 V, 2200 mAh, 30C) – simplified datasheet view

Li-Po 3S Battery (11.1 V, 2200 mAh, 30C)	
Type: Lithium-Polymer (Li-Po) Rechargeable Battery	
Nominal Voltage: 11.1 V (3 Cells in series, 3.7 V per cell)	
Maximum Voltage: 12.6 V (Fully charged at 4.2 V per cell)	
Capacity: 2200 mAh (2.2 Ah)	
Discharge Rate: 30C Continuous (Capable of safely delivering up to 66 A of continuous current)	
Connectors: XT60 or T-Dean main power plug ; JST-XH balance plug for charging	
Safety Notes: Requires a dedicated balance charger. Must not be discharged below 3.2 V per cell to prevent permanent chemical damage.	

4.4.3. Voltage Regulation:

To safely power the STM32 microcontroller, Laser, and Ultrasonic sensors, we utilized an XL4015 DC-DC Step-Down Converter. It efficiently steps down the main battery voltage (11.1V) to a clean, regulated 5V, featuring high conversion efficiency and built-in thermal shutdown protection.

Converter:

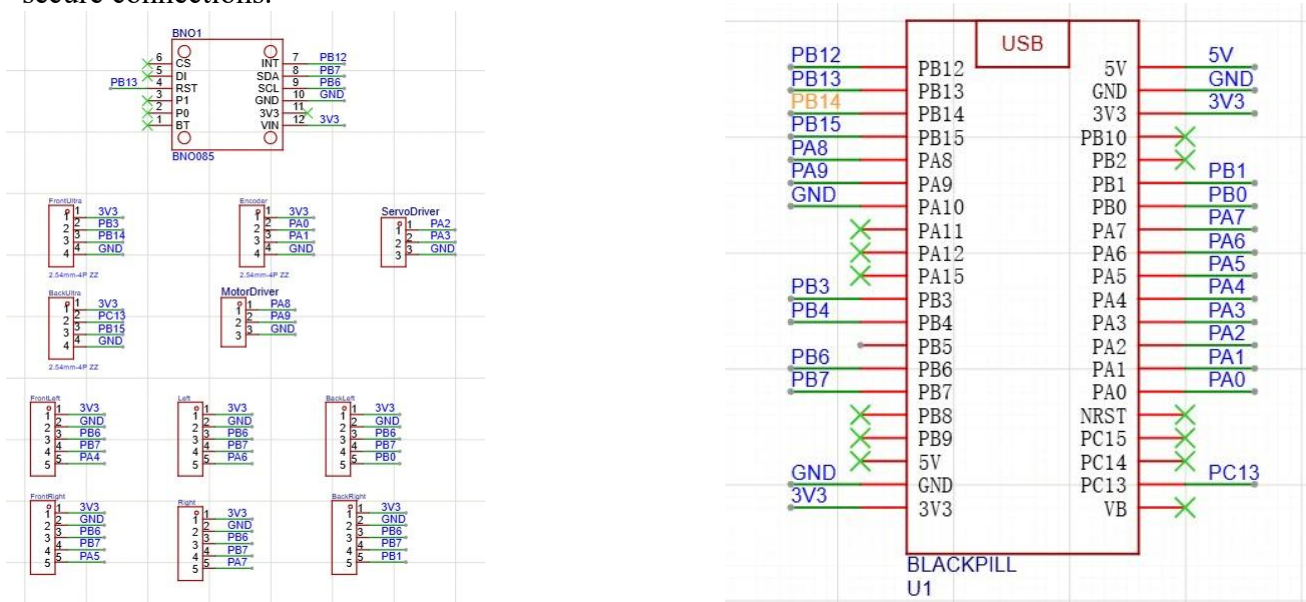
XL4015 DC-DC Step-Down Converter
Type: High-Efficiency DC-DC Step-Down (Buck) Converter
Input Voltage Range: 8.0 V – 36.0 V (Perfectly accommodates the 11.1 V 3S Li-Po battery)
Output Voltage Range: 1.25 V – 32.0 V Adjustable (Calibrated to a stable 5.0 V for logic and sensors)
Output Current: Up to 5.0 A maximum (with a heatsink)
Switching Frequency: 180 kHz
Efficiency: Up to 96%
Protection Features: Built-in thermal shutdown, short-circuit protection, and current limiting



Table 12: XL4015 DC-DC Step-Down Converter – simplified datasheet view.

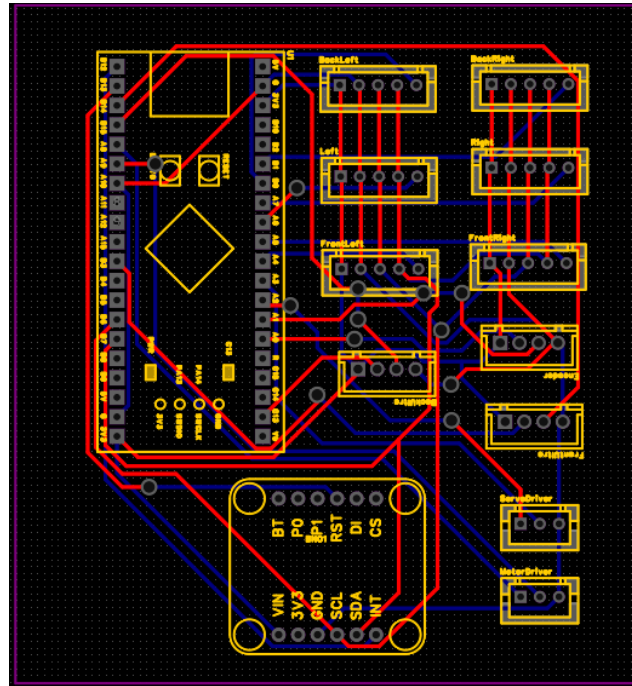
4.5. Custom Printed Circuit Board (PCB) To eliminate the unreliability of loose jumper wires and breadboards, we designed a custom Printed Circuit Board (PCB). The PCB acts as the central nervous system of the robot's hardware, cleanly routing power and data between the STM32 microcontroller, the sensor arrays (VL53L0X and US-100), the motor drivers (IBT-4 and Waveshare), and the XL4015 power management module.

4.5.1. Schematic Design The schematic was engineered to ensure complete isolation between the logic circuits and the high-current actuators. It includes properly sized traces for power delivery, pull-up resistors for the I2C lines (IMU and Laser sensors), and organized header pins for direct, secure connections.



4.5.2. PCB Layout & Routing The physical board layout was designed with careful consideration for electromagnetic interference (EMI) and power dissipation. The XL4015 step-down converter and the STM32 BlackPill are strategically placed to minimize signal travel distance. Ground pours (Copper Pours) were utilized extensively across the board to provide a common ground reference and dissipate any excess heat.

4.5.3.



5. Obstacle Management

Our navigation system relies on real-time sensor evaluation and precise image classification to smoothly guide the robot through obstacles and successfully complete the three laps.

5.1. System Architecture & Communication

Instead of using heavy middleware, we adopted a lightweight, ultra-fast Master-Slave architecture relying on direct serial communication (UART Protocol) between the processing units:

5.1.1. Master Node:

Represented by the Raspberry Pi 5 running a Python script. It captures frames, processes them via OpenCV, and determines strategic commands (e.g., `TURN_LEFT` or `DODGE_RIGHT`).

5.1.2. Execution Node:

Represented by the STM32 microcontroller. It receives these short string commands and immediately integrates them into a Non-blocking Finite State Machine (FSM) to update motor speeds and PID offsets, without halting distance or IMU sensor readings. This approach ensures high-frequency data flow with near-zero latency.

5.2. Open Challenge

In this challenge, the goal is to complete 3 laps without hitting the walls in the shortest possible time. We rely on a wall-following algorithm using ultrasonic sensors, minimizing the error between the right and left distances to zero using a PID controller.

5.2.1. "Direction Lock" Algorithm:

The camera does not rely on colors to determine the turn direction at every corner; instead, it uses a smart locking mechanism. Upon starting, the camera monitors the floor; if it detects an Orange line first, the system locks the direction as "Clockwise" (CW). If a Blue line is detected, it locks as "Counter-Clockwise" (CCW). For the subsequent 11 corners, the robot executes the turn based on this locked direction as soon as any color is seen, entirely preventing visual confusion and ensuring track stability.

5.3. Image Processing

To avoid color fluctuations due to lighting, we convert the frames captured by the tilt-down camera from the RGB format to the HSV color space.

5.3.1. Region of Interest & Contouring:

We apply masks to isolate the four colors (Blue, Orange, Red, Green), utilize morphological filters to remove noise, and extract Bounding Boxes for each element to determine its position and dimensions on the screen.

5.4. Obstacle Challenge

While corners rely on the pixel density of the floor lines, pillars require high spatial accuracy to determine dodging priority. To overcome complex scenarios (like having both red and green pillars in the same frame), we developed the Y-Axis Depth Sorting algorithm:

The algorithm collects the coordinates of all detected pillars and sorts them in descending order based on the lowest point of their bounding box (y_{bottom}).

All distant pillars are ignored, and the system interacts only with the pillar having the highest y value (the one closest to the front of the robot).

If the closest pillar is Red, the Raspberry Pi sends a DODGE_RIGHT command. The STM32 translates this command into a dynamic bias ($Lane_Offset$) within the distance sensors' PID equation, smoothly hugging the right wall. For a Green pillar, DODGE_LEFT is sent. This dynamic maneuver allows the robot to perform professional, continuous Slalom movements without needing to stop.

5.5. Performance Videos

These are the links to the YouTube videos of our robot successfully completing the challenges:

5.5.1. Open Challenge:

[\[Link\]](#)

5.5.2. Obstacle Challenge:

[\[Link\]](#)

5.6. Problems We Encountered

5.6.1. Camera Field of View Limitation

5.6.1.1. Problem: In the early development stages, we used the standard Raspberry Pi camera. However, its narrow Field of View (FOV) made it difficult for the robot to see the full width of the track or detect peripheral pillars in a timely manner.

5.6.1.2. Solution: We replaced it with the FANTECH LUMINOUS C30 web camera. This camera features a wide-angle lens and crisp resolution, enabling the computer vision algorithms to capture all surrounding details and recognize corners and obstacles from a safe, early distance.

5.6.2. Ackermann Steering Angle Limitations

5.6.2.1. Problem: During maneuver testing and tight pillar passing, we encountered limitations in the maximum steering angle achievable by our initial Ackermann steering mechanism. This increased the robot's turning radius, dangerously pushing it towards the outer walls.

5.6.2.2. Solution: We redesigned the steering knuckles and tie rods using SolidWorks and 3D-printed them with geometrically optimized pivot points and lever lengths. This adjustment significantly increased the Range of

Motion for the front wheels, allowing the robot to make sharp, large-angle turns smoothly while maintaining the correct Ackermann intersection to prevent tire scrub.

5.6.3. Flickering

5.6.3.1. Problem: Variations in room lighting and the frequency of fluorescent lamps caused flickering in the frames captured by the camera, which negatively impacted the stability of our color masks.

5.6.3.2. Solution: We disabled Auto White Balance and programmatically adjusted the camera's shutter speed and exposure via OpenCV to synchronize with the ambient lighting frequency, resulting in perfectly stable and clean frames.

5.6.4. Failing to Synchronize Data Flow Between Processors

5.6.4.1. Problem: The Raspberry Pi sent a continuous, heavy stream of data over serial (UART) to the STM32, leading to buffer overflow and a delayed motor response to camera detections.

5.6.4.2. Solution: We abandoned continuous streaming and designed a lightweight communication protocol that sends short, event-based commands (e.g., `TURN_LEFT`) only when the state changes. We also implemented a non-blocking Finite State Machine (FSM) in the STM32 to ensure instantaneous serial commands are read without halting other vital operations.

5.6.5. Pillars Contours Merging

5.6.5.1. Problem: In track sections with closely placed pillars, the low camera perspective caused the pillars' contours to merge in the frame. The algorithm misidentified them as a single large obstacle, leading to traversal failures.

5.6.5.2. Solution: We modified the mechanical design by elevating the camera to the third layer of the robot and optimizing its tilt-down angle. This elevation provided excellent visual separation between the pillars and allowed our Y-Axis Depth Sorting algorithm to function with pinpoint accuracy.

6. Improvements and future modifications

6.1. Mobility Management

6.1.1. Aluminum CNC Upgrade:

Although the 3D-printed parts are lightweight and currently serve their purpose, we plan to replace the Ackermann steering knuckles and motor mounts with CNC-machined aluminum parts. This will eliminate any mechanical flex in the joints at high speeds, thereby enhancing steering precision and extending the chassis' lifespan.

6.1.2. Differential Upgrade:

We aim to replace the current differential with a fully sealed metal differential containing lubricating fluid to reduce internal friction and power loss. This upgrade will significantly improve the efficiency of torque transfer from the Ga25 motor to the silicone wheels.

6.2. Power and Sense Management

6.2.1. Battery Management System (BMS):

We plan to integrate a smart BMS module into our custom PCB. This unit will monitor individual cell voltages of the Li-Po battery and the real-time current draw. This data will be transmitted via UART to the Raspberry Pi, allowing the system to dynamically cap the motor's top speed to prevent battery fatigue or sudden voltage drops.

6.2.2. LiDAR Integration:

Despite the efficiency of the VL53L0X and US-100 sensors, upgrading to a 2D LiDAR sensor (e.g., RPLIDAR) will provide a 360-degree point cloud map. This will reduce the number of discrete sensors required and offer unparalleled accuracy for the robot's localization within the track.

6.3. Obstacle Management

6.3.1. Deep Learning for Vision:

Given the massive processing power of our Raspberry Pi 5 (8GB), we plan to upgrade our image processing pipeline. Instead of relying on OpenCV HSV color masks, we aim to deploy lightweight Deep Learning models (such as YOLO). This will make the detection system completely immune to drastic lighting changes or color distortions on the pillars.

6.3.2. Optimization of the Selected Path:

According to WRO rules, pillar placements remain fixed across all three laps. We plan to develop a "Path Caching" algorithm during the relatively slower first lap to memorize the colors and coordinates of the pillars. In the second and third laps, the robot will utilize this memory to navigate the obstacles at maximum speed without evaluating the camera frames from scratch in every section.

7. Conclusion

Throughout this project, our goal was never simply to build a robot that completes the track, but to engineer a system where every component was chosen for a defensible reason and every subsystem was validated through testing rather than assumption.

The defining decision of our design was the **Hybrid Control Architecture**: separating high-level perception on the Raspberry Pi 5 from hard real-time execution on the STM32F411. This split allowed each processor to do what it does best — computer vision without timing pressure, and deterministic PID control without operating-system jitter — and directly resolved the serial buffer overflow that had been delaying our motor response to camera detections.

Equally important was our commitment to iterative refinement. Nearly every subsystem in this report exists in its current form because an earlier version failed under test. Our steering knuckles were redesigned in SolidWorks after we hit the limits of our original Ackermann geometry. Our camera moved to the third layer only after merged pillar contours revealed that a low perspective was costing us spatial separation. Our communication protocol was rewritten from continuous streaming to event-based commands after buffer overflows appeared. Each of these problems taught us more than a first-attempt success would have.

Beyond the technical outcomes, this project developed our capabilities in mechanical design, PCB layout, embedded firmware, computer vision, and — critically — the systems thinking required to make these disciplines work together. The improvements outlined in Section 6 represent our clear roadmap forward, and we are confident that the foundation documented here is robust enough to support them.

We are proud of what team X-CUTE has built, and we look forward to competing in WRO 2026.